

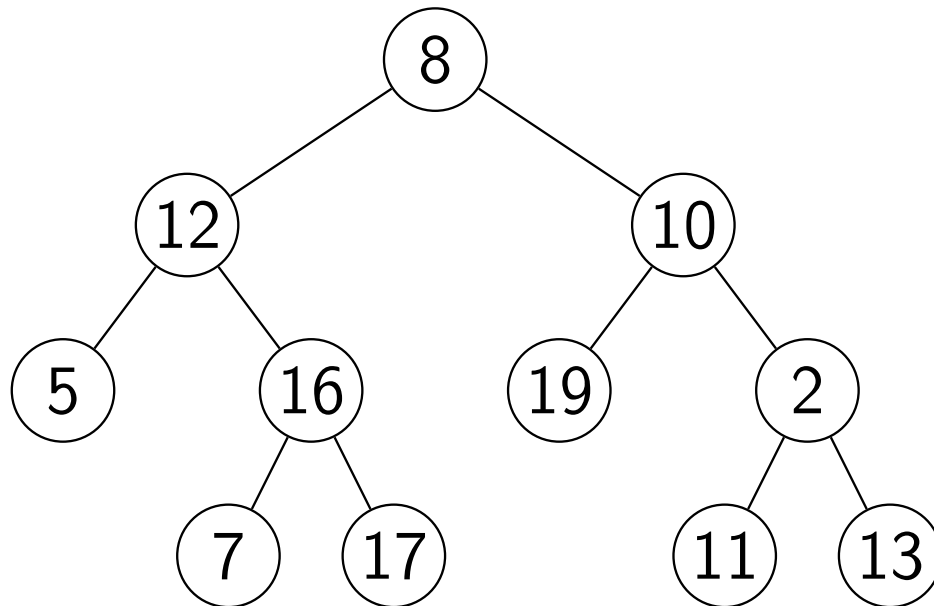
Partie 2

Réduire l'imbrication des structures de contrôle

Description d'un problème

On considère le problème suivant : étant donné un arbre enraciné, écrire les valeurs de chaque nœud, dans l'ordre de lecture, chaque niveau de l'arbre sur une ligne différente.

Exemple :



Sortie :

```
8
12 10
5 16 19 2
7 17 11 13
```

Une solution peu satisfaisante

Exercice

Consulter et essayer de comprendre la solution proposée dans le fichier cours.`BreadthFirstSearchV1.java`

Imbrication

- L'**imbrication** est le fait d'utiliser des structures de contrôles (une boucle **for** ou **while**, une structure conditionnelle **if**) les unes dans les autres.
- On peut ainsi avoir une boucle dans une boucle dans une boucle...dans une méthode.
- Chaque “dans une” correspond à un degré supplémentaire d'imbrication.

Pour garder une méthode lisible, il est essentiel de ne pas avoir un degré d'imbrication élevé (idéalement 0 : pas de structure de contrôle, ou 1 : pas d'imbrication).

Comment simplifier l'algorithme

- Extraire les **boucles internes** pour en faire des **méthodes** ;
- Séparer les boucles complexes en plusieurs, idéalement chaque boucle fait **une seule chose** ;
- Soigner le choix des **noms** des méthodes.

Consulter les étapes de refactorisation :

- `cours1.BreadthFirstSearchV2` ;
- `cours1.BreadthFirstSearchV3` ;
- `cours1.BreadthFirstSearchV4`.

Règles d'écriture des méthodes

- Commencer par une description à haut-niveau, **ensuite** aller vers les détails.
- Ne pas hésiter à introduire de nombreuses méthodes : cela vous force à **nommer** et donc à réfléchir au **rôle** de chaque méthode, et cela rend le programme plus facile à lire et à vérifier.
- **Éviter les boucles imbriquées** : elles sont difficiles à comprendre. Sortez la boucle interne dans une méthode.
- L'IDE peut vous assister dans l'**extraction de méthode**.
- Suivre les principaux **patrons de méthodes**.

Les patrons de méthodes

Liste des patrons de méthodes

Les méthodes répétant des instructions ont l'un de ces patrons :

- Itérer une opération sur tous les éléments ;
- Réduire à une seule valeur ;
- Chercher une valeur ;
- Répéter tant que nécessaire.

Patron de méthode : itérer une opération

Faire une opération sur tout ou partie des éléments d'une collection :

```
public void iterateProcess(Collection<Elt> elements) {  
    initialise();  
    for (Elt element : elements) {  
        if (element.isGood()) {  
            process(element);  
        }  
    }  
    return;  
}
```

une boucle **for** contenant si besoin une conditionnelle **if**.

Patron de méthode : réduction

Faire un calcul agrégeant les éléments d'une collection :

```
public ReturnType aggregate(Collection<Elt> elements) {  
    ReturnType value = initialise();  
    for (Elt element : elements) {  
        value = process(value, element);  
    }  
    return value;  
}
```

Une boucle **for**, quelques instructions dont une affectation.

Patron de méthode : tester une collection

Tester que tous les éléments d'une liste vérifient une condition :

```
public boolean areAllGood(Collection<Elt> elements) {  
    for (Elt element : elements) {  
        if (!element.isGood()) {  
            return false;  
        }  
    }  
    return true;  
}
```

Un **for**, un **if**, on quitte la méthode dès qu'un mauvais élément est trouvé (**return**).

Patron de méthode : boucle `while`

Répéter une opération jusqu'à la réalisation d'une condition :

```
public void repeatProcess() {  
    value = initialise();  
    while (isNotOver(value)) {  
        instructions; // value must change!  
    }  
    return;  
}
```

Cas classique : traitement d'une file d'attente :

```
while (!waitingList.isEmpty()) {  
    E element = waitingList.pollFirst();  
    process(element, waitingList);  
}
```

Exemple

Consulter les classes suivantes, illustrant comment un programme peut gagner en lisibilité grâce à de bonnes pratiques de programmation, **en essayant d'anticiper les améliorations** :

- `cours1.SieveV1`
- `cours1.SieveV2`
- `cours1.SieveV3`
- `cours1.SieveV4`
- `cours1.SieveV5`

Quitter une boucle

- `break` et `return` permettent de quitter immédiatement la boucle en cours.
- `continue` permet de mettre fin à l'itération en cours de la boucle. La boucle ne termine pas, l'itération suivante commence (si la condition de continuation est satisfaite).

Drapeau booléen

L'usage d'un drapeau booléen, une variable qui contrôle la continuation de la boucle, tend à rendre les méthodes **illisibles**. Il faut **préférer** l'usage de **break** et **continue**.

Consulter les exemples (en essayant d'anticiper les améliorations) :

- `cours1.InteractiveSumV1;`
- `cours1.InteractiveSumV2;`
- `cours1.InteractiveSumV3;`
- `cours1.InteractiveSumV4.`

Quand utiliser des boucles imbriquées ?

Il est légitime d'utiliser des boucles imbriquées pour :

- du calcul matriciel : les matrices sont en deux dimensions, une boucle par dimension,
- d'autres objets bidimensionnels : images, grille, ...

```
for (int column = 0; column < nbColumns; column++) {  
    for (int row = 0; row < nbRows; row++) {  
        ...  
    }  
}
```